Análisis crítico comparativo de ClickHouse como sistema de gestión de bases de datos analítico distribuido frente a Apache Cassandra

Estudiante: Luis Jeremy Jaramillo Acosta | **Paralelo:** A | **Actividad:** TA-IND-01

Universidad Técnica Estatal de Quevedo — Aplicaciones Distribuidas — Noviembre de 2026

Resumen

Este informe presenta un análisis técnico y crítico de ClickHouse, un sistema de gestión de bases de datos orientado a columnas diseñado para procesamiento analítico en línea (OLAP) distribuido, comparándolo con la arquitectura de fila ancha particionada de Apache Cassandra. Se evalúa el funcionamiento interno de ClickHouse a partir de los fundamentos de la Unidad 1: grados de transparencia de distribución, ubicación dentro del teorema CAP y la extensión PACELC, protocolos de comunicación entre nodos (sockets TCP binarios y coordinación mediante consenso Raft en ClickHouse Keeper) y mecanismos de tolerancia a fallos sobre tablas `MergeTree` replicadas. Para comprobar estos conceptos en la práctica, se desplegó un entorno local con contenedores Docker donde se midió la ingestión por lotes de un millón de registros, el volumen de datos procesados y la respuesta del clúster ante la detención abrupta de un nodo. Por último, se relacionan estos compromisos arquitectónicos con plataformas reales utilizadas en el entorno local ecuatoriano y se argumenta por qué ClickHouse sobresale en consultas analíticas masivas a costa de sacrificar operaciones transaccionales punto a punto.

1. Introducción

El aumento acelerado de registros de telemetría, trazas de red y eventos digitales ha dejado en evidencia las limitaciones de las bases de datos relacionales monolíticas cuando deben calcular agregaciones sobre cientos de millones de filas en tiempo real [1]. Mientras los motores transaccionales (OLTP) se enfocan en insertar, indexar y actualizar registros individuales con rapidez, las cargas analíticas necesitan recorrer únicamente unas pocas columnas a través de volúmenes masivos de datos sin bloquear las operaciones concurrentes.

ClickHouse ataca este problema mediante una arquitectura distribuida de almacenamiento columnar compartido nada (*shared-nothing*). Nació originalmente para impulsar la plataforma de analítica web Yandex.Metrica y hoy sostiene cargas críticas en empresas como Cloudflare (procesando millones de solicitudes HTTP por segundo para detectar ataques y métricas de tráfico), Uber (almacenamiento de registros operativos), eBay y Deutsche Bank. Su motor permite ingerir millones de filas por segundo por servidor y resolver consultas sobre petabytes de información en fracciones de segundo.

La elección de ClickHouse y su contraste con Apache Cassandra responde a que ambos sistemas resuelven el escalamiento horizontal de grandes volúmenes de datos bajo filosofías opuestas dentro de los sistemas distribuidos [2]: ClickHouse apuesta por fragmentación (*sharding*) con ejecución vectorizada y coordinación fuerte de metadatos mediante consenso [3], mientras Cassandra emplea un anillo descentralizado *peer-to-peer* con hashing consistente pensado para garantizar disponibilidad continua de escritura.

2. Arquitectura del Sistema

ClickHouse opera bajo un diseño *shared-nothing* en el que cada nodo del clúster administra su propio procesador, memoria RAM y almacenamiento local en disco, eliminando cuellos de botella asociados a volúmenes compartidos [1], [2]. Su estructura interna se divide en cuatro capas acopladas:

1. **Capa de Acceso y Coordinación de Consultas:** Cualquier nodo puede recibir consultas SQL a través de su protocolo binario nativo sobre TCP (puerto 9000) o mediante su interfaz HTTP/REST (puerto 8123). Cuando la consulta se dirige a una tabla virtual con motor `Distributed`, ese nodo actúa como coordinador: descompone el plan de ejecución, envía subconsultas en paralelo a todos los fragmentos (*shards*) y unifica los resultados parciales antes de responder al cliente.

2. **Motor de Almacenamiento Columnar (`MergeTree`):** En lugar de guardar filas completas contiguas, ClickHouse escribe cada columna en un archivo binario independiente ordenado por una clave primaria dispersa en memoria. Cada inserción por lotes crea un directorio inmutable llamado *parte* (`part`), comprimido con algoritmos LZ4 o ZSTD. En segundo plano, hilos asíncronos fusionan (*merge*) partes pequeñas en bloques más grandes y ordenados.

3. **Fragmentación (*Sharding*) y Replicación:** Los conjuntos de datos se particionan horizontalmente en múltiples *shards* para distribuir tanto el almacenamiento como la carga de CPU [4]. Dentro de cada *shard*, el motor `ReplicatedMergeTree` mantiene copias sincronizadas apoyándose en un registro replicado de operaciones para evitar pérdida de datos [3].

4. **Plano de Control y Consenso (*ClickHouse Keeper*):** La coordinación entre réplicas, la ejecución distribuida de sentencias DDL y la asignación de nombres de partes fusionadas recaen sobre *ClickHouse Keeper*. Este servicio, desarrollado en C++, implementa el algoritmo de consenso **Raft** (puerto TCP 9181) y sustituye la antigua dependencia externa de Apache ZooKeeper [2], [3].

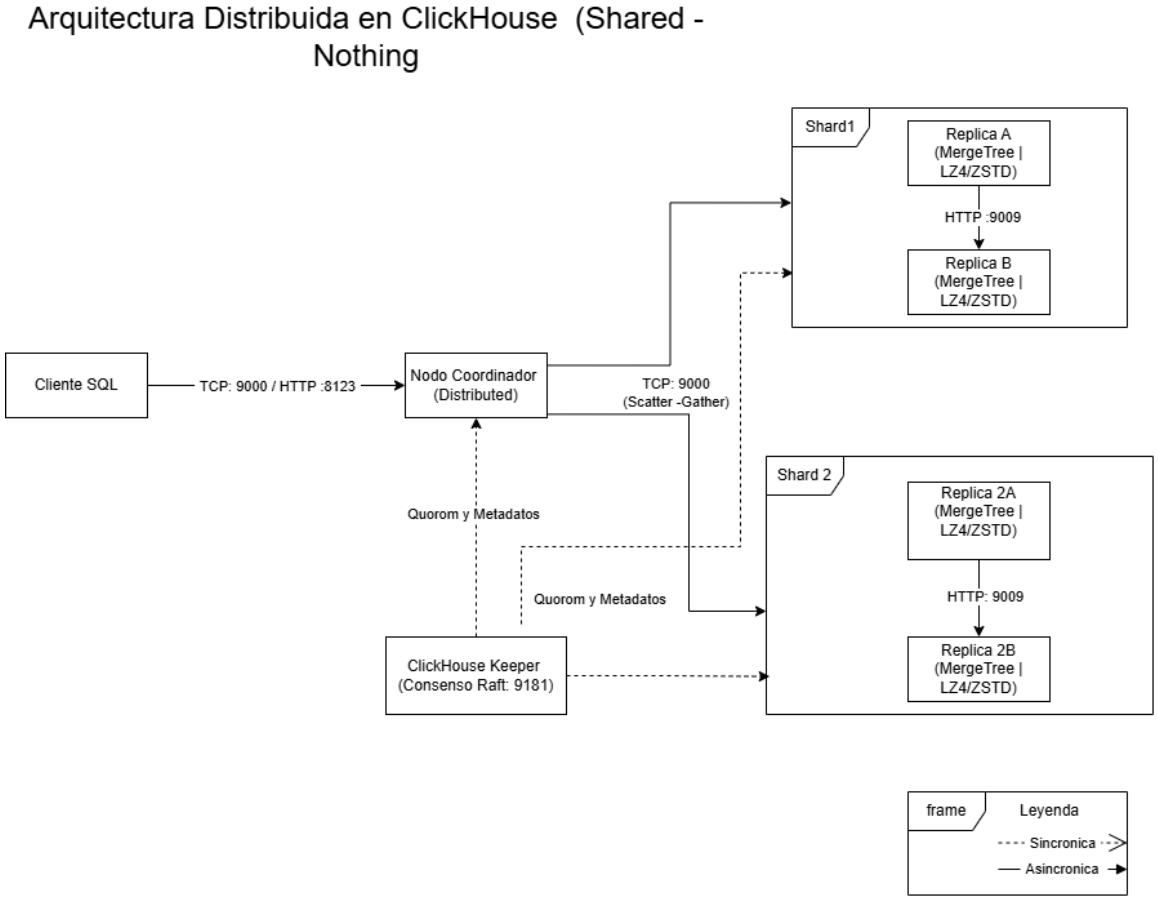


Figura 1: Arquitectura distribuida de ClickHouse: nodos coordinadores, shards con almacenamiento columnar MergeTree y plano de consenso ClickHouse Keeper (Raft). Elaboración propia.

La Figura 1 recoge la evolución técnica respecto al bosquejo inicial trazado a mano (Anexo A), diferenciando el plano de datos (flujo columnar masivo y replicación por puerto 9009) del plano de control (consenso Raft sobre metadatos).

3. Análisis con los Conceptos de la Unidad 1

3.1. Transparencias de Distribución

Tomando como referencia la clasificación de van Steen y Tanenbaum [2], ClickHouse prioriza aquellas transparencias que no comprometen el rendimiento de lectura analítica:

- **Transparencia de Acceso (Alta):** El cliente ejecuta consultas SQL estándar sin preocuparse por cómo se serializan las columnas en binario ni qué esquema de compresión se aplica en disco, ya sea contra un nodo aislado o un clúster de decenas de máquinas.

- **Transparencia de Ubicación (Alta):** Al consultar una tabla con motor `Distributed`, la aplicación interactúa con un único nombre lógico de tabla; el coordinador resuelve internamente las direcciones IP y puertos de cada *shard* [2].
- **Transparencia de Replicación (Alta):** En tablas `ReplicatedMergeTree`, basta con insertar los datos en una réplica para que el sistema propague los bloques hacia las demás copias del mismo *shard*; en lectura, el coordinador enruta la petición hacia una réplica disponible [3].
- **Transparencia de Concurrencia (Alta):** Gracias al control de concurrencia multiversión (MVCC) sobre partes inmutables, las consultas de lectura toman una instantánea consistente (*snapshot isolation*) sin bloquear las inserciones entrantes ni verse frenadas por las fusiones en segundo plano.
- **Transparencia de Fallos (Alta en entornos replicados):** Ante la caída de una réplica, las consultas se redirigen automáticamente hacia otra réplica saludable del *shard*. Cuando el nodo afectado vuelve en línea, recupera las partes faltantes desde la bitácora de consenso sin intervención manual [3].
- **Transparencia de Migración y Reubicación (Limitada):** ClickHouse no rebalancea datos automáticamente en caliente al añadir nuevos servidores. Reubicar particiones entre *shards* exige comandos administrativos explícitos (`move partition`), ya que mover terabytes de forma automática saturaría el ancho de banda y degradaría las consultas analíticas.

3.2. Teorema CAP y Extensión PACELC

Frente a una partición de red (**P**) dentro del teorema CAP, ClickHouse se comporta como un sistema predominantemente **CP** en su plano de control, aunque permite ajustar el comportamiento de escritura según las necesidades del negocio [2], [3]:

- *ClickHouse Keeper* (consenso Raft) requiere mayoría estricta (quórum) para aprobar cambios de esquema y registrar nuevas partes replicadas [3]. Si un nodo queda aislado en el lado minoritario de una partición sin acceso al quórum, sus tablas replicadas pasan a modo de solo lectura para impedir divergencias de metadatos (*split-brain*).
- No obstante, con la configuración predeterminada (`insert_quorum = 0`), un nodo que conserve conectividad con Keeper puede aceptar inserciones en su disco local y diferir la replicación hasta que se restablezca el enlace con sus pares.

Al evaluar el sistema bajo la extensión **PACELC** (que analiza el compromiso entre latencia **L** y consistencia **C** durante la operación normal sin particiones **E**), ClickHouse funciona por defecto en modo **PC/EL**: ante particiones protege la consistencia del plano de control (**PC**), pero en condiciones normales privilegia la baja latencia (**EL**) replicando los bloques de forma asíncrona para responder rápido al cliente [4]. Si una aplicación crítica exige lecturas estrictamente linealizables, se puede activar `insert_quorum` y `select_sequential_consistency` para operar como **PC/EC**. En cambio, Apache Cassandra está diseñado de origen como **PA/EL**.

3.3. Modelos de Comunicación

ClickHouse combina tres esquemas de comunicación entre procesos (IPC) adaptados al tipo de tráfico que transportan [2], [5], [6]:

1. **Cliente-Servidor sobre Sockets TCP Binarios (Puerto 9000):** Para transferir consultas y bloques de columnas comprimidas, el cliente nativo utiliza un protocolo binario orientado a columnas sobre conexiones TCP persistentes. Como evidencian los estudios comparativos entre protocolos binarios e interfaces textuales en sistemas distribuidos [5], [6], evitar la serialización en texto JSON/REST disminuye drásticamente la carga de CPU y la latencia. De forma complementaria, ofrece una API HTTP/REST (puerto 8123) para integración directa con herramientas web.
2. **Comunicación Nodo-a-Nodo (Coordinador ↔ Shards y Réplicas):** Durante una consulta distribuida, el coordinador abre sockets TCP binarios en paralelo hacia los nodos que alojan cada *shard* bajo un patrón síncrono de dispersión-recolección (*scatter-gather*). Por el contrario, la copia de partes entre réplicas de un mismo *shard* se realiza de manera **asíncrona** mediante un servidor HTTP interno (puerto 9009).
3. **Consenso Distribuido (Sockets TCP / Protocolo Raft):** Los nodos de datos mantienen conexiones TCP persistentes con el clúster de *ClickHouse Keeper* utilizando un protocolo binario compatible con ZooKeeper respaldado por consenso Raft [3].

3.4. Tolerancia a Fallos

La arquitectura soporta caídas abruptas (*crash-stop*) de nodos réplica, cortes temporales de red, pérdida del líder de consenso y corrupción de bloques en disco [2], [3]:

- **Detección:** Cada nodo envía mensajes periódicos de latido (*heartbeats*) hacia *ClickHouse Keeper*. Si transcurre el tiempo límite (`session_timeout_ms`) sin respuesta, Keeper elimina el nodo efímero de sesión y marca la réplica como no disponible. En el plano de almacenamiento, cada archivo columnar guarda sumas de verificación (*checksums*) que permiten detectar sectores dañados antes de procesar la consulta.
- **Recuperación:** La bitácora replicada de Keeper mantiene el orden exacto de inserciones y fusiones confirmadas, aplicando el mismo principio de registro anticipado de escritura replicado (WAL) analizado en bases de datos distribuidas modernas [3]. Al reiniciar el nodo caído, este contrasta su inventario local de partes contra la bitácora de Keeper, descarga únicamente las partes faltantes desde las réplicas activas y vuelve a recibir tráfico sin pérdida de información confirmada.
- **Incidente real documentado:** En despliegues productivos de gran escala (como los casos documentados por el equipo de ingeniería de Cloudflare y los reportes operativos de ClickHouse), un fallo recurrente ocurre cuando aplicaciones productoras envían miles de inserciones individuales por segundo sin agruparlas en lotes. Esto provoca una explosión de carpetas inmutables pequeñas que satura las colas de metadatos de ZooKeeper/Keeper y los hilos de fusión en disco; el motor activa entonces un mecanismo de contrapresión rechazando nuevas escrituras con el error *"Too many parts"* hasta que los procesos de *merge* consolidan los bloques y estabilizan el nodo.

4. Experimentación Propia

Para contrastar la teoría con mediciones reales, se montó un entorno experimental local con contenedores Docker ejecutando la imagen oficial `clickhouse/clickhouse-server` sobre

Windows. Tanto los comandos SQL (`experimento_clickhouse.sql`) como las salidas de consola (`resultados_experimento.txt`) quedaron registrados en el repositorio.

4.1. Desarrollo de las Pruebas y Evidencias de Ejecución

1. **Despliegue de múltiples nodos en Docker (Figura 2):** Se iniciaron dos contenedores independientes (`ch-nodo1` en los puertos 8123/9000 y `ch-nodo2` en los puertos 8124/9001), registrando mediante `whoami`, `hostname` y `get-date` la ejecución directa en el equipo local.
2. **Creación de esquema columnar particionado (Figura 3):** A través del cliente nativo TCP (`clickhouse-client`), se creó la base de datos `analitica_uteq` y la tabla `eventos_red` con motor `MergeTree()`, particionada por mes (`partition by toYYYYMM(fecha)`) y ordenada por `(origen, id_evento)`.
3. **Ingestión por lotes y consulta de agregación (Figura 4):** Se insertó un lote de **1 000 000 de registros sintéticos** mediante `numbers(1000000)` en **0,403 segundos** (8,00 MB procesados a 2,48 millones de filas/s), simulando mediciones de latencia entre dos puntos (`nodo-valencia` y `nodo-quevedo`), y se ejecutó una consulta analítica con `count()`, `avg()` y `max()` resuelta en **0,030 segundos**.
4. **Cierre de sesión nativa y simulación de caída de nodo (Figura 5):** Se finalizó la sesión del cliente TCP mediante el comando `quit` y se detuvo abruptamente el segundo contenedor con `docker stop ch-nodo2` para verificar la continuidad operativa del nodo primario.

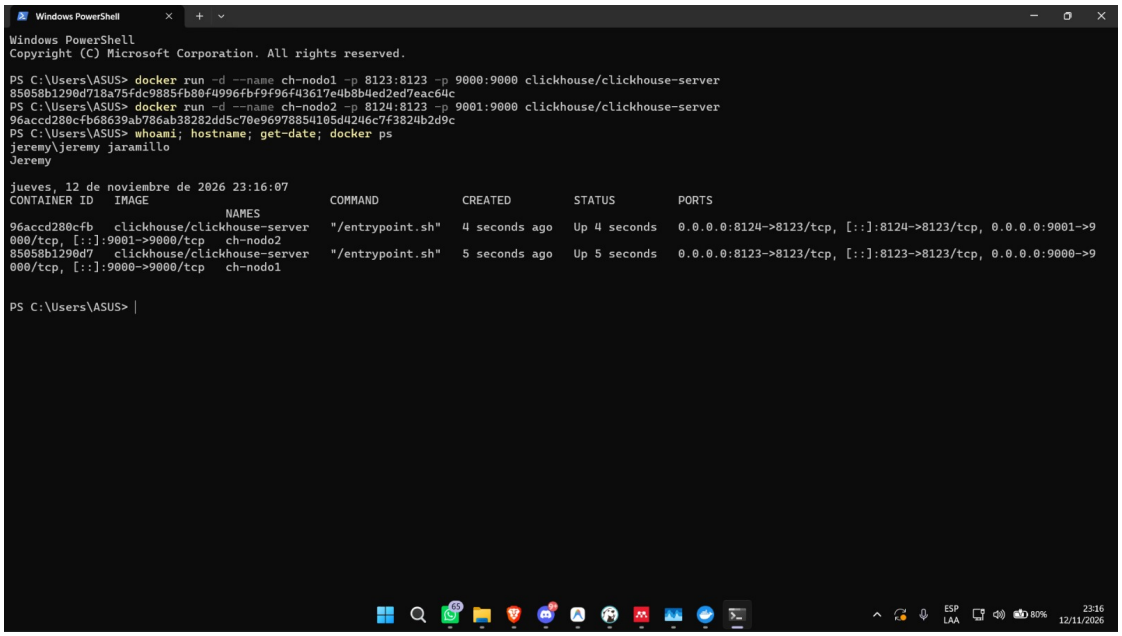


Figura 2: Despliegue de los contenedores `ch-nodo1` y `ch-nodo2` en Docker con verificación de usuario y fecha del sistema.

4.2. Interpretación Técnica de Resultados

Las mediciones obtenidas validan tres principios clave del diseño de ClickHouse:

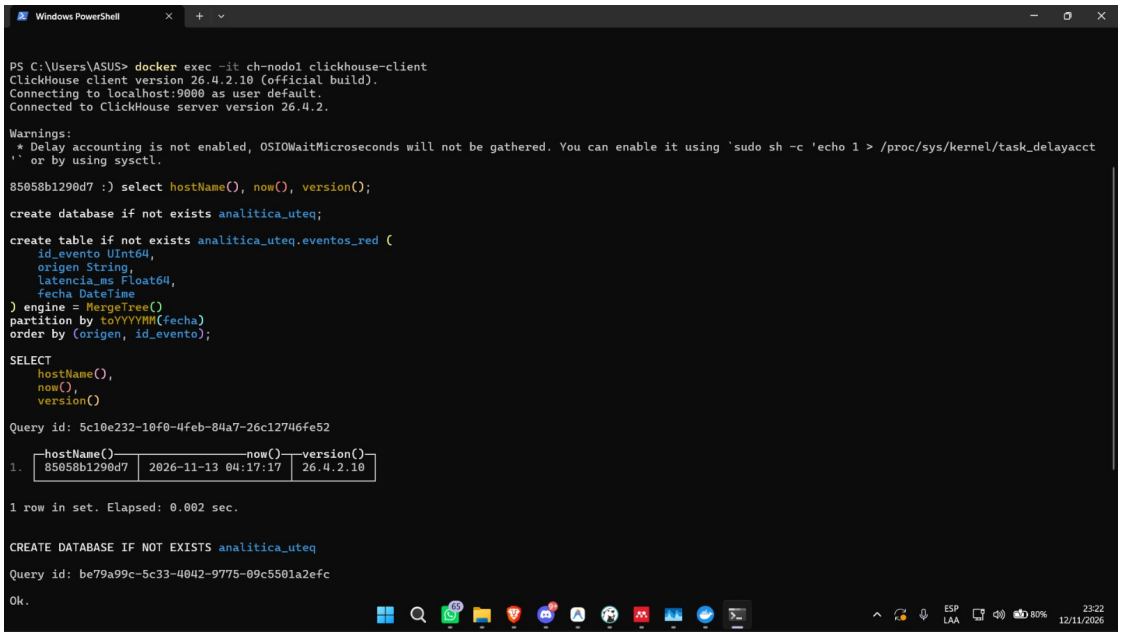


Figura 3: Creación de la base de datos `analitica_uteq` y de la tabla columnar `eventos_red` con motor `MergeTree()`.

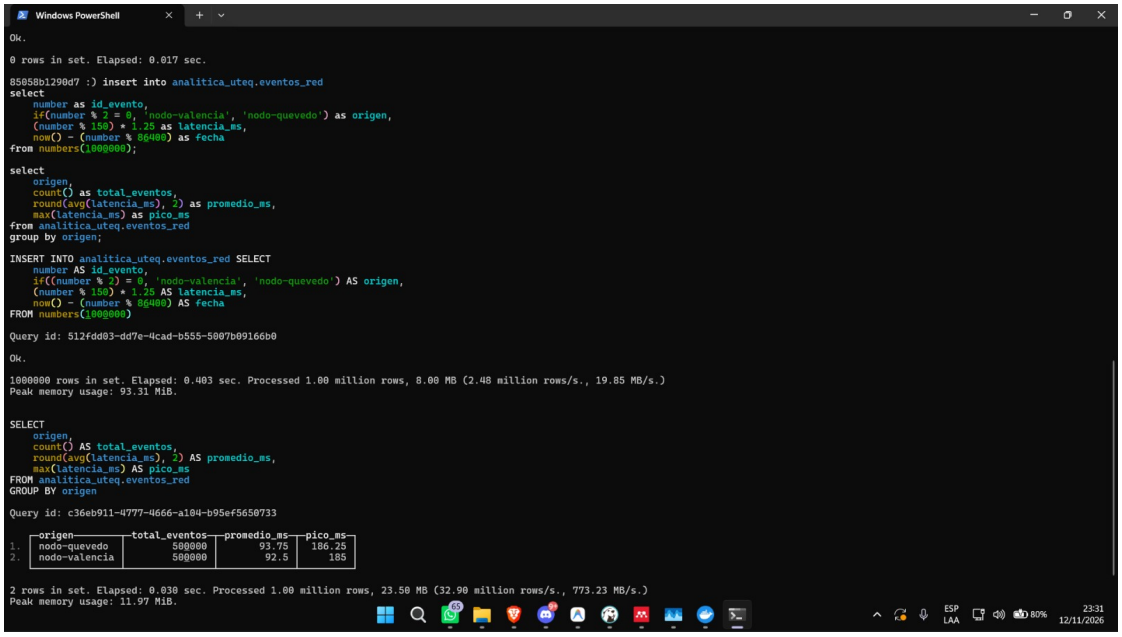


Figura 4: Inserción por lotes de 1 000 000 de registros en 0,403 s y ejecución de consulta analítica de agregación en 0,030 s.

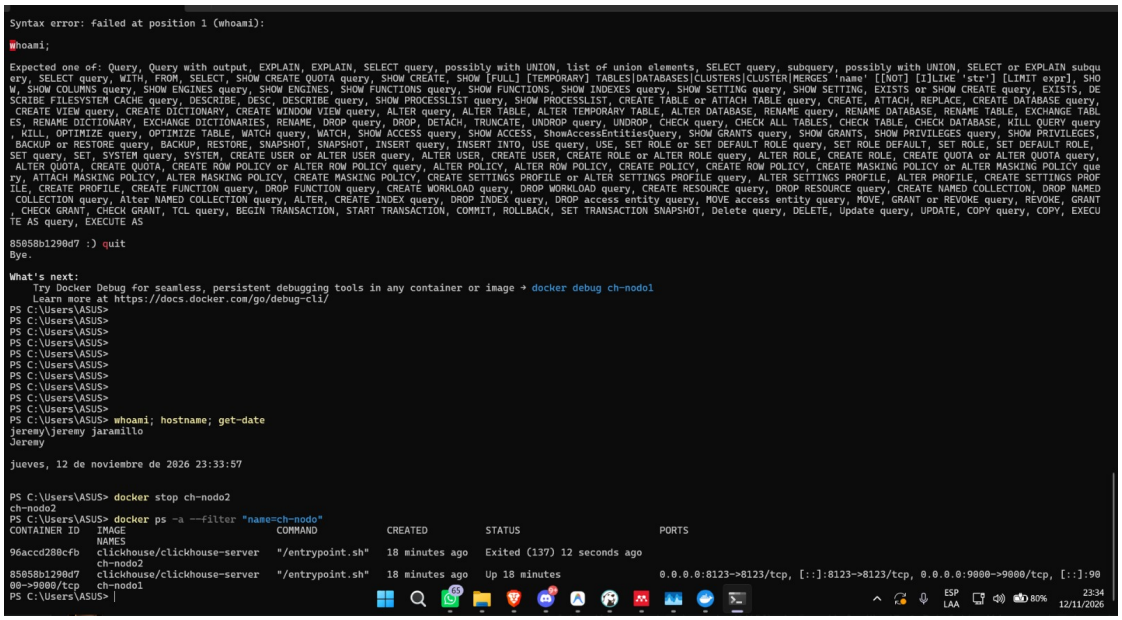


Figura 5: Cierre de sesión en `clickhouse-client` y simulación de caída de nodo mediante `docker stop ch-nodo2`.

- **Lectura selectiva por columnas:** Al calcular `count()`, `avg(latencia_ms)` y `max(latencia_ms)` agrupados por `origen`, el motor leyó exclusivamente los archivos de esas dos columnas e ignoró por completo `id_evento` y `fecha`, resolviendo el millón de filas en **0,030 segundos** (32,90 millones de filas/s) con un consumo pico de apenas **11,97 MiB** de memoria RAM.
- **Ingestión secuencial de alta velocidad (*append-only*):** La inserción por lotes de un millón de registros se completó en **0,403 segundos** procesando apenas **8,00 MB**, lo que confirma la eficiencia de escribir bloques ordenados de forma secuencial sin bloqueos de fila ni validaciones transaccionales individuales [3].
- **Aislamiento bajo arquitectura *shared-nothing*:** Tras forzar la detención de `ch-nodo2` (estado `Exited (137)`), el nodo `ch-nodo1` continuó activo (`Up`) sin interrupciones al no depender de memoria ni almacenamiento compartido bloqueante.

5. Ejemplos del Entorno del Estudiante

Los compromisos entre consistencia, disponibilidad y concurrencia de la Unidad 1 también se observan en servicios digitales de uso diario en Ecuador:

1. **Banca Móvil y Pagos Inmediatos (Banco Pichincha / Aplicación De Una):** Al realizar transferencias o pagos cotidianos mediante la aplicación móvil de Banco Pichincha y la billetera De Una entre Valencia y Quevedo durante horas pico (quincenas o fines de mes), la aplicación confirma de inmediato el comprobante de débito, pero el historial de movimientos o la notificación de acreditación al destinatario demora entre varios segundos y un par de minutos. Este comportamiento refleja la **consistencia eventual** y la prioridad de baja latencia (**EL**) del modelo PACELC [2], [4]: el núcleo transaccional registra el movimiento crítico sin bloquear al usuario y propaga de manera asíncrona las vistas de consulta y notificaciones.

2. **Sistema de Gestión Académica (SGA) de la UTEQ:** Durante los periodos de matriculación ordinaria y cierre de actas en la Universidad Técnica Estatal de Quevedo (UTEQ), cuando cientos de estudiantes ingresan simultáneamente a revisar cupos e inscribir materias, el portal sufre demoras severas, cierres de sesión o errores temporales de conexión. Desde la ingeniería de sistemas distribuidos, esto evidencia una limitación en la **transparencia de concurrencia y escalabilidad horizontal** [1], [2]: al concentrar consultas masivas y escrituras transaccionales sobre una base de datos centralizada sin separación de cargas analíticas, la contención de bloqueos degrada la disponibilidad del portal.

6. Análisis Comparativo

La Tabla 1 compara sistemáticamente **ClickHouse** frente a **Apache Cassandra** bajo siete criterios técnicos de la Unidad 1.

6.1. Discusión del Contraste Arquitectónico

La diferencia central entre ambos motores reside en cómo organizan el almacenamiento en disco y coordinan el estado del clúster. Apache Cassandra prescinde de coordinadores centrales mediante su topología *peer-to-peer* y protocolo Gossip, garantizando escrituras ininterrumpidas incluso frente a particiones de red severas; sin embargo, al agrupar los datos por clave de partición de fila, las consultas analíticas que agregan millones de registros sin filtrar por esa clave (`sum`, `avg`, `group by`) obligan a recorrer filas completas en múltiples nodos, disparando la latencia. ClickHouse sigue el camino inverso: acepta la complejidad de mantener consenso Raft en *ClickHouse Keeper* [3] y restringe las actualizaciones fila a fila, pero logra velocidades de consulta analítica órdenes de magnitud superiores mediante lectura columnar secuencial y procesamiento vectorizado SIMD.

Tabla 1: Comparación arquitectónica entre ClickHouse y Apache Cassandra según los criterios de la Unidad 1.

Criterio	ClickHouse	Apache Cassandra
1. Modelo de datos y almacenamiento	Orientado a columnas (`MergeTree`); almacena cada columna en archivos comprimidos separados ordenados por clave primaria dispersa.	Orientado a fila ancha particionada (*Wide-Column*); utiliza estructuras LSM-Tree (*MemTable* + *SSTables*) indexadas por clave de partición.
2. Topología y modelos de comunicación	Clúster *shared-nothing* con nodos coordinadores y *shards*; sockets TCP binarios (:9000), HTTP (:8123) y consenso Raft (:9181) [3], [5].	Anillo descentralizado *Peer-to-Peer* sin nodo maestro; hashing consistente, protocolo Gossip (:7000) y protocolo binario CQL (:9042) [2].
3. Clasificación CAP y PACELC	**CP** en metadatos (Keeper/Raft) y **PC/EL** por defecto en datos (configurable a **PC/EC** con quórum de escritura y lectura).	**AP** ante particiones y **PA/EL** por defecto (consistencia sintonizable por consulta mediante niveles `ONE`, `QUORUM`, `ALL`).
4. Transparencias de distribución	Alta en acceso, ubicación (`Distributed`), replicación y concurrencia; limitada en migración automática de *shards*.	Muy alta en ubicación, replicación, migración (rebalanceo automático de tokens en el anillo) y tolerancia a fallos [2].
5. Tolerancia a fallos y recuperación	Réplicas coordinadas por bitácora Raft en ClickHouse Keeper; descarga puntual de partes faltantes y validación por *checksums* [3].	Sin punto único de fallo; utiliza *Hinted Handoff*, reparación en lectura (*Read Repair*) y sincronización anti-entropía con árboles de Merkle.
6. Mutabilidad y patrón de escritura	Optimizado para inserciones masivas por lotes (*append-only*); las actualizaciones individuales (`alter table update`) son pesadas y asíncronas.	Optimizado para escrituras aleatorias de alta velocidad (*upsert*); cada modificación escribe una nueva marca de tiempo sin lectura previa.
7. Carga de trabajo óptima	Procesamiento analítico en línea (OLAP), agregaciones complejas, telemetría, logs y series temporales masivas.	Aplicaciones operacionales distribuidas (OLTP masivo), mensajería, catálogos y sistemas que exigen disponibilidad continua de escritura [4].

7. Postura Crítica

Evaluado con criterio de ingeniería de software, ClickHouse demuestra una especialización muy marcada con ventajas claras y compromisos inevitables [1]:

- **Fortalezas:** La combinación de ejecución vectorizada, compresión columnar nativa y control de concurrencia sin bloqueos permite responder consultas analíticas complejas en milisegundos utilizando una fracción del hardware que exigirían motores orientados a filas. De igual manera, sustituir ZooKeeper por *ClickHouse Keeper* simplifica notablemente la administración del clúster.
- **Limitaciones y compromisos (*trade-offs*):** No ofrece transacciones ACID multi-fila con bloqueos pesimistas, no valida unicidad (`UNIQUE`) ni llaves foráneas (`FOREIGN KEY`) al insertar, y penaliza severamente las inserciones unitarias de una sola fila debido a la fragmentación de partes en disco.
- **Contextos donde NO debe utilizarse:** Resulta inadecuado como base de datos transaccional principal (OLTP) en sistemas bancarios centrales, control de inventario en tiempo real o sistemas de reservas donde múltiples usuarios modifican simultáneamente registros individuales con aislamiento estricto (escenarios propios de PostgreSQL o motores transaccionales distribuidos), así como en redes intermitentes distribuidas geográficamente que demanden escritura continua sin quórum (donde Apache Cassandra es superior) [1], [4].

8. Conclusiones

1. La velocidad analítica de ClickHouse responde directamente a la sinergia entre el almacenamiento físico columnar inmutable (`MergeTree`), la compresión por bloques y la lectura selectiva de atributos, hecho verificado en el experimento local al insertar 1 000 000 de registros en 0,403 segundos (8,00 MB procesados) y resolver su agregación analítica en apenas 0,030 segundos.
2. La incorporación de consenso Raft en *ClickHouse Keeper* y la replicación ordenada de bloques ubican a ClickHouse como un sistema **PC/EL** bajo el modelo PACELC, asegurando consistencia fuerte en metadatos y baja latencia en la ingestión masiva de datos [2], [3].
3. El contraste con Apache Cassandra confirma que toda arquitectura distribuida responde a un compromiso de diseño [1]: Cassandra maximiza la disponibilidad de escritura y el rebalanceo automático mediante su anillo *peer-to-peer*, mientras ClickHouse domina el análisis masivo (OLAP) a cambio de exigir ingestión por lotes y limitar las mutaciones puntuales.

Declaración de Uso de Inteligencia Artificial

En cumplimiento de las normas de integridad académica de la asignatura, se declara que para la elaboración de este informe se utilizó la herramienta de inteligencia artificial generativa **Google Gemini** (versión 2026) como apoyo editorial y de formato. Su uso se limitó exclusivamente a: (a) estructuración de la plantilla en LaTeX y configuración del estilo bibliográfico IEEE mediante `biblatex`, (b) revisión ortográfica, gramatical y de estilo en la redacción del documento, y (c)

verificación de coherencia estructural frente a los criterios de la rúbrica. Toda la experimentación en contenedores Docker y las consultas SQL fueron diseñadas y ejecutadas directamente por el estudiante en su equipo local, los ejemplos del entorno corresponden a experiencias reales propias y cada referencia bibliográfica fue comprobada manualmente mediante su respectivo identificador DOI o ISBN en las publicaciones originales.

Referencias Bibliográficas

[1] P. I. Habib, A. S. Murdha, H. Agus y Suhardi, «Architecture Migration From Monolithic to Microservices: Developing Readiness Criteria,» *IEEE Access*, vol. 12, págs. 194 630-194 645, 2024. DOI: 10.1109/ACCESS.2024.3504848

[2] M. van Steen y A. S. Tanenbaum, *Distributed Systems*, 4.ª ed. Leiden, The Netherlands: Maarten van Steen, 2023, ISBN: 978-9081540636. dirección: https://www.distributed-systems.net/index.php/books/ds4/

[3] F. Han et al., «PALF: Replicated Write-Ahead Logging for Distributed Databases,» *Proceedings of the VLDB Endowment*, vol. 17, n.º 12, págs. 3745-3758, 2024. DOI: 10.14778/3685800.3685803

[4] S. Newman, *Building Microservices: Designing Fine-Grained Systems*, 2.ª ed. O'Reilly Media, Inc., 2021, ISBN: 978-1492033974.

[5] I. Buzhin, A. Derevyankin, V. Antonova, A. Perevalov e Y. Mironov, «Comparative Analysis of the REST and gRPC Used in the Monitoring System of Communication Network Virtualized Infrastructure,» *T-Comm*, vol. 17, n.º 4, págs. 50-55, 2023. DOI: 10.36724/2072-8735-2023-17-4-50-55

[6] G. Blinowski, A. Ojdowska y A. Przybylek, «Monolithic vs. Microservice Architecture: A Performance and Scalability Evaluation,» *IEEE Access*, vol. 10, págs. 20 357-20 374, 2022. DOI: 10.1109/ACCESS.2022.3152803

A. Anexo A: Esquema Inicial de Arquitectura a Mano

A continuación se presenta el bosquejo inicial elaborado a mano que sirvió como punto de partida para modelar la interacción entre el cliente SQL, el nodo coordinador, los fragmentos (*shards*) con almacenamiento columnar y el servicio de consenso *ClickHouse Keeper*.

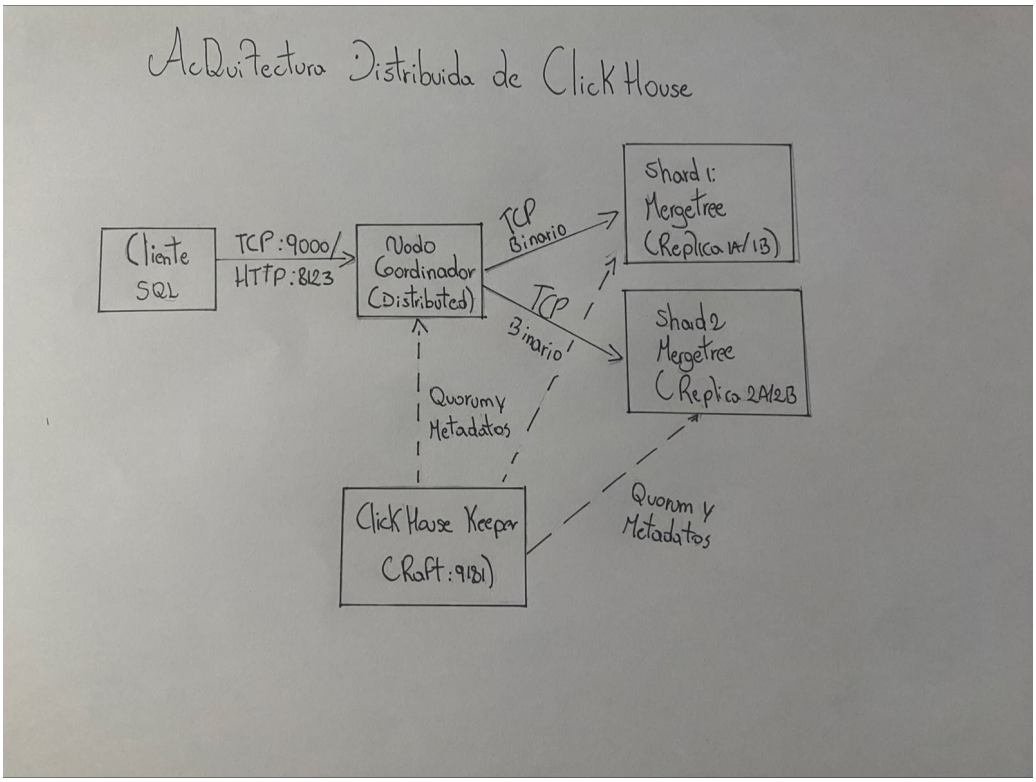


Figura 6: Bosquejo inicial a mano de la arquitectura distribuida de ClickHouse con firma y fecha del estudiante.

B. Anexo B: Capturas de Lectura y Anotación de Fuentes

Se adjuntan las evidencias de lectura directa, subrayado y anotación crítica de tres artículos científicos indexados citados en el desarrollo del informe:

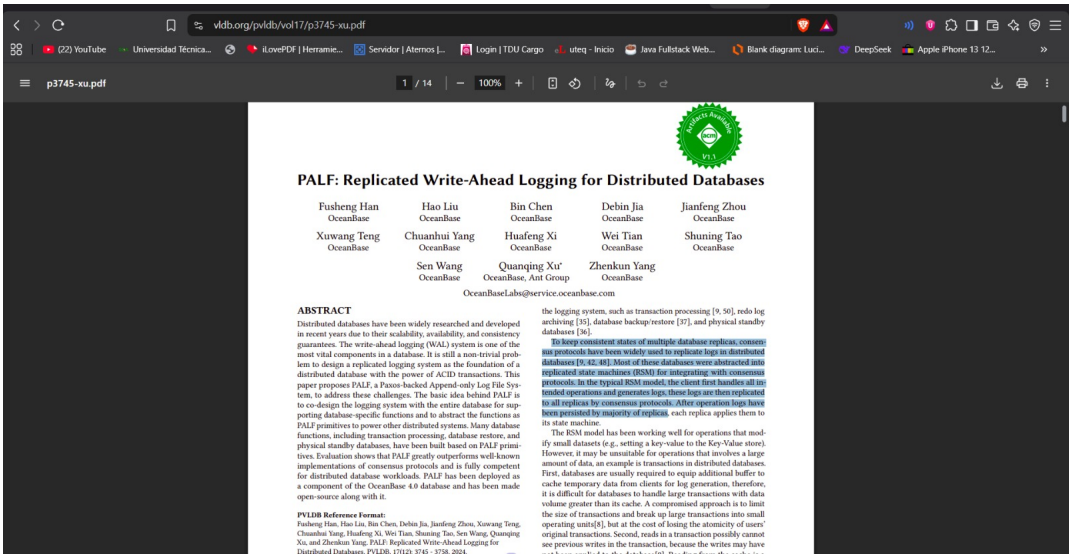


Figura 7: Fragmento subrayado y anotado de Han et al. (2024), *Proceedings of the VLDB Endowment*, vol. 17, no. 12, p. 3745, sobre protocolos de consenso y replicación de bitácoras WAL en bases de datos distribuidas [3].

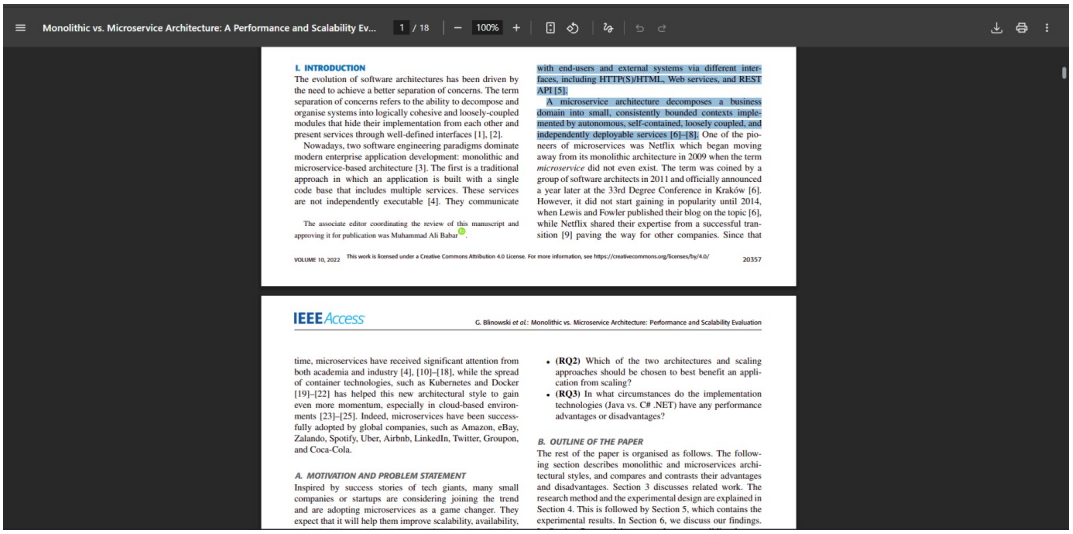


Figura 8: Fragmento subrayado de Blinowski et al. (2022), *IEEE Access*, vol. 10, p. 20357, sobre descomposición en servicios autónomos desacoplados e interfaces de comunicación en red [6].

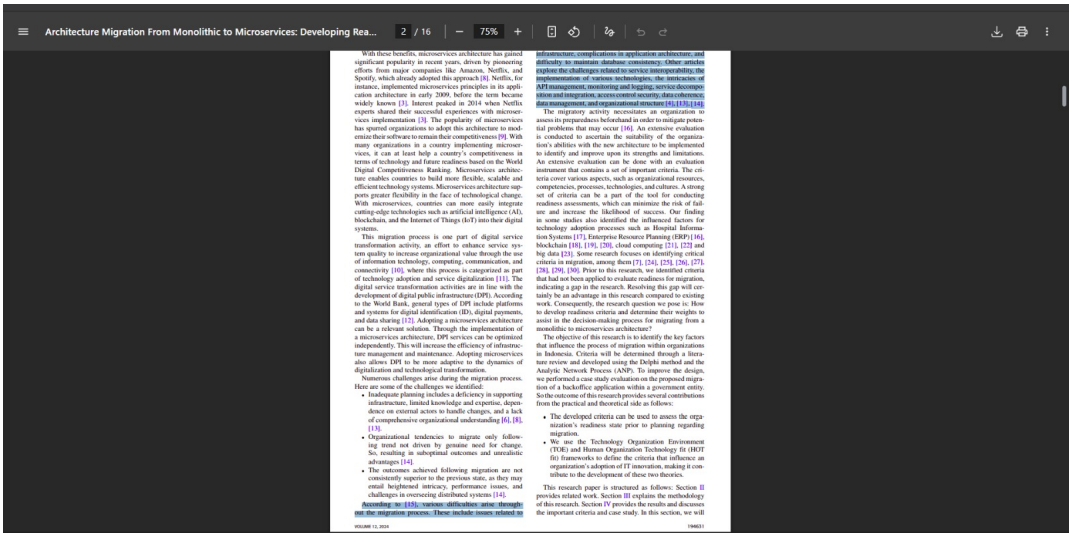


Figura 9: Fragmento subrayado y anotado de Habib et al. (2024), *IEEE Access*, vol. 12, p. 194631, sobre desafíos de consistencia de bases de datos y coherencia de datos en entornos distribuidos [1].